

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 11



Disusun Oleh:

Wisnu Bimantoro 105224045

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. Pendahuluan

Pada praktikum Modul 11 saya mempelajari konsep Error Handling pada Java. Materi ini membahas cara menangani kesalahan program menggunakan try-catch-finally, throw, throws, serta pembuatan custom exception. Dengan adanya exception handling, program dapat tetap berjalan dengan baik dan memberikan informasi kesalahan yang jelas kepada pengguna.

Pada THT ini saya membuat program Sistem Reservasi Tiket Kereta Java Express. Program digunakan untuk menampilkan jadwal kereta, melakukan reservasi tiket, memvalidasi data penumpang, serta menangani berbagai kondisi kesalahan seperti NIK tidak valid, kode kereta tidak ditemukan, dan jumlah tiket yang melebihi kursi yang tersedia.

II. Variabel

No	Variabel	Tipe Data	Fungsi
1	kodeKereta	String	Menyimpan kode kereta
2	namaKereta	String	Menyimpan nama kereta
3	rute	String	Menyimpan rute perjalanan
4	sisaKursi	int	Menyimpan jumlah kursi tersedia
5	daftarKereta	ArrayList<Kereta>	Menyimpan seluruh data kereta
6	kodeKereta	String	Parameter pencarian kereta
7	nik	String	Menyimpan NIK penumpang
8	nama	String	Menyimpan nama penumpang
9	jumlahTiket	int	Menyimpan jumlah tiket yang dipesan
10	keretaDipilih	Kereta	Menyimpan objek kereta hasil pencarian
11	input	Scanner	Membaca input pengguna
12	pilihan	int	Menyimpan pilihan menu

III. Constructor dan Method

No	Nama Method	Jenis	Fungsi
1	Kereta(...)	Constructor	Mengisi data kereta
2	getKodeKereta())	Method	Mengambil kode kereta
3	getNamaKereta() a()	Method	Mengambil nama kereta
4	getRute()	Method	Mengambil rute
5	getSisaKursi()	Method	Mengambil sisa kursi
6	setSisaKursi()	Method	Mengubah sisa kursi
7	ReservasiService() e()	Constructor	Menginisialisasi data kereta
8	tampilkanJadwal() al()	Method	Menampilkan jadwal kereta
9	pesanTiket()	Method	Melakukan reservasi tiket
10	main()	Static Method	Titik awal program

IV. Pembahasan Code

Struktur Program:

1. Kereta.java → Menyimpan data kereta.
2. ReservasiService.java → Mengelola proses reservasi.
3. DataPenumpangTidakValidException.java → Exception NIK tidak valid.
4. RuteTidakDitemukanException.java → Exception kode kereta tidak ditemukan.
5. TiketHabisException.java → Exception tiket habis.
6. Main.java → Menjalankan program.

1. Penjelasan Class Kereta

```
public class Kereta {  
  
    private String kodeKereta;  
    private String namaKereta;  
    private String rute;
```

```

private int sisaKursi;

public Kereta(String kodeKereta,
               String namaKereta,
               String rute,
               int sisaKursi) {

    this.kodeKereta = kodeKereta;
    this.namaKereta = namaKereta;
    this.rute = rute;
    this.sisaKursi = sisaKursi;
}
}

```

Penjelasan:

Pada bagian ini dibuat class Kereta yang berfungsi sebagai model atau representasi data kereta. Di dalam class terdapat empat atribut yaitu kodeKereta, namaKereta, rute, dan sisaKursi.

Constructor digunakan untuk menginisialisasi seluruh atribut ketika objek kereta dibuat.

Keyword this digunakan untuk membedakan atribut milik class dengan parameter constructor yang memiliki nama sama.

Logika utama class ini adalah menyimpan seluruh informasi yang berkaitan dengan kereta. Setiap objek yang dibuat akan memiliki identitas dan data yang berbeda sehingga dapat digunakan oleh sistem reservasi untuk menampilkan jadwal dan melakukan pemesanan tiket.

Pembahasan Getter dan Setter

```

public String getKodeKereta() {
    return kodeKereta;
}

public String getNamaKereta() {
    return namaKereta;
}

public String getRute() {
    return rute;
}

public int getSisaKursi() {
    return sisaKursi;
}

```

```

    }

    public void setSisaKursi(int sisaKursi) {
        this.sisaKursi = sisaKursi;
    }

```

Penjelasan

Method getter digunakan untuk mengambil nilai atribut dari class Kereta.

Method setter digunakan untuk memperbarui nilai sisaKursi ketika terjadi pemesanan tiket.

Saat pengguna melakukan reservasi, jumlah kursi yang tersedia harus berkurang. Oleh karena itu program membutuhkan setter untuk memperbarui data kursi yang tersimpan pada objek kereta.

2. Pembahasan Class DataPenumpangTidakValidException

```

public class DataPenumpangTidakValidException
    extends RuntimeException {

    public DataPenumpangTidakValidException(String pesan) {
        super(pesan);
    }
}

```

Pembahasan:

Class ini merupakan custom exception yang dibuat sendiri oleh programmer.

Class mewarisi RuntimeException sehingga termasuk kategori Unchecked Exception.

Method super(pesan) digunakan untuk mengirimkan pesan error ke parent class.

Exception ini akan muncul ketika pengguna memasukkan NIK yang tidak sesuai aturan, misalnya jumlah digit kurang dari 16 atau mengandung huruf.

Dengan adanya exception khusus ini, pesan kesalahan menjadi lebih jelas dibandingkan menggunakan exception bawaan Java..

3. Pembahasan Class RuteTidakDitemukanException

```
public class RuteTidakDitemukanException
    extends Exception {

    public RuteTidakDitemukanException(String pesan) {
        super(pesan);
    }
}
```

Pembahasan

Class ini merupakan custom checked exception karena mewarisi class Exception.

Exception akan dipaksa untuk ditangani menggunakan try-catch atau throws.

Apabila pengguna memasukkan kode kereta yang tidak tersedia pada sistem, program akan menghentikan proses reservasi dan memberikan informasi bahwa kereta tidak ditemukan.

4. Penjelasan Class TiketHabisException

```
public class TiketHabisException extends Exception {

    private String namaKereta;
    private int sisaKursi;

    public TiketHabisException(String namaKereta,
                                int sisaKursi) {

        super("Tiket tidak mencukupi.");

        this.namaKereta = namaKereta;
        this.sisaKursi = sisaKursi;
    }
}
```

Penjelasan

Class ini dibuat untuk menangani kondisi ketika jumlah tiket yang dipesan lebih banyak daripada jumlah kursi yang tersedia.

Selain menyimpan pesan error, class ini juga menyimpan nama kereta dan jumlah kursi yang tersisa.

Dengan menyimpan data tambahan, sistem dapat memberikan informasi yang lebih lengkap kepada pengguna mengenai kondisi kursi yang tersedia.

5. Pembahasan Constructor ReservasiService

```
public ReservasiService() {  
  
    daftarKereta = new ArrayList<>();  
  
    daftarKereta.add(  
        new Kereta(  
            "K01",  
            "Argo Bromo",  
            "JKT - SBY",  
            50));  
  
    daftarKereta.add(  
        new Kereta(  
            "K02",  
            "Parahyangan",  
            "JKT - BDG",  
            15));  
}
```

Penjelasan Kode

Pada constructor ini dibuat objek ArrayList untuk menyimpan data kereta.

Kemudian ditambahkan dua objek kereta sebagai data awal sistem.

Saat program dijalankan, sistem harus memiliki data kereta yang dapat dipilih oleh pengguna. Oleh karena itu data langsung dimasukkan ke dalam ArrayList pada constructor.

Pembahasan Method tampilkanJadwal()

```

for (Kereta kereta : daftarKereta) {

    System.out.println(
        kereta.getKodeKereta()
        + " | "
        + kereta.getNamaKereta()
        + " | "
        + kereta.getRute()
        + " | Sisa Kursi : "
        + kereta.getSisaKursi());
}

```

Penjelasan Kode

Program menggunakan enhanced for loop untuk membaca seluruh objek kereta yang ada di dalam ArrayList.

Data setiap kereta kemudian ditampilkan ke layar.

Karena jumlah kereta bisa lebih dari satu, diperlukan perulangan untuk menampilkan seluruh data tanpa harus menulis kode berulang.

Pembahasan Method pesanTiket()

```

if (!nik.matches("\\d{16}")) {

    throw new DataPenumpangTidakValidException(
        "NIK harus terdiri dari 16 digit angka.");
}

```

Penjelasan Kode

Method matches() digunakan untuk mencocokkan input dengan pola regex.

Pola \\d{16} berarti hanya menerima 16 digit angka.

NIK merupakan identitas penting sehingga harus divalidasi terlebih dahulu sebelum proses reservasi dilakukan.

Kode Pencarian Kereta

```

for (Kereta kereta : daftarKereta) {

    if (kereta.getKodeKereta()
        .equalsIgnoreCase(kodeKereta)) {

```



```

        keretaDipilih = kereta;
        break;
    }
}

```

Penjelasan Kode

Program mencari kereta berdasarkan kode yang dimasukkan pengguna.

Jika ditemukan maka objek disimpan ke variabel keretaDipilih.

Reservasi hanya bisa dilakukan jika kereta yang dipilih memang tersedia dalam sistem.

Kode Validasi Kursi

```

if (jumlahTiket >
    keretaDipilih.getSisaKursi()) {

    throw new TiketHabisException(
        keretaDipilih.getNamaKereta(),
        keretaDipilih.getSisaKursi());
}

```

Penjelasan Kode

Program membandingkan jumlah tiket yang diminta dengan jumlah kursi yang tersedia.

Pemesanan tidak boleh melebihi kapasitas kursi yang tersedia agar data reservasi tetap valid.

6. Pembahasan Class Main

```

System.out.println("1. Lihat Jadwal");
System.out.println("2. Pesan Tiket");
System.out.println("3. Keluar");

```

Penjelasan Kode

Kode digunakan untuk menampilkan pilihan menu kepada pengguna.

Menu berfungsi sebagai navigasi utama agar pengguna dapat memilih fitur yang ingin digunakan.

Kode Try-Catch

```

try {

    reservasi.pesanTiket(
        kode,
        nik,
        nama,
        jumlah);

}
catch (DataPenumpangTidakValidException e) {

    System.out.println(e.getMessage());
}

```

Penjelasan Kode

Blok try digunakan untuk menjalankan proses yang berpotensi menghasilkan exception.

Blok catch digunakan untuk menangkap exception yang terjadi.

Dengan exception handling, program tidak langsung berhenti ketika terjadi kesalahan input, melainkan memberikan pesan yang lebih mudah dipahami pengguna.

Kode Finally

```

finally {

    if (pilihan == 3) {

        System.out.println(
            "Sistem reservasi ditutup.");
    }
}

```

Penjelasan Kode

Blok finally akan selalu dijalankan baik terjadi error maupun tidak.

Bagian ini memastikan sistem tetap menampilkan pesan penutupan program ketika pengguna keluar dari aplikasi.

V. Kesimpulan

Kesimpulan dari praktikum Modul 11 ini adalah saya berhasil memahami penerapan Error Handling pada Java menggunakan try-catch-finally, throw, throws, serta custom exception. Melalui program Sistem Reservasi Tiket Kereta Java Express, saya dapat memahami bagaimana menangani kesalahan input dan kondisi khusus tanpa membuat program berhenti secara tiba-tiba. Penerapan exception handling membuat program menjadi lebih aman, mudah dipelihara, dan memberikan informasi kesalahan yang lebih jelas kepada pengguna.

VI. Daftar Pustaka

Deitel, P. J., & Deitel, H. M. (2017). *Java How to Program, Early Objects* (11th ed.). Pearson. <https://www.pearson.com/en-us/subject-catalog/p/java-how-to-program-early-objects/P200000003310/9780134743356>

Farrell, J. (2022). *Java Programming* (10th ed.). Cengage Learning. <https://www.cengage.com/c/java-programming-10e-farrell/9780357675137/>

Oracle. (n.d.). *Control Flow Statements*. The Java™ Tutorials. <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/flow.html>

Tim Asisten Praktikum. (2026). *Modul 3: Java Basics II*. Laboratorium Komputer Program Studi Ilmu Komputer, Universitas Pertamina.

W3Schools. (n.d.). *Java If...Else*. https://www.w3schools.com/java/java_conditions.asp